

SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

SystemCraft Platform

Comprehensive Documentation of the High Level Design (HLD) & Low Level Design (LLD) Simulation, Learning, and Architecture Platform

PROJECT CODENAME

SystemCraft (TechSim)

ARCHITECTURE VERSION

v3.2 Production

STACK

**React 19, Express 5, MongoDB,
Groq/Gemini**

TARGET COMPLETION

Unified HLD & LLD Ecosystem

Table of Contents

1. **Executive Summary & System Overview** — Platform mission, engineering philosophy, and full tech stack.
2. **Complete Feature Catalog & Business Logic Deep-Dive** — Detailed breakdown of all 14 platform subsystems, physics models, and code implementations.
3. **Current Status & Feature Health Audit** — Real-time validation of all routes, APIs, and resolved bugs.
4. **High Level Design (HLD) Playground: Roadmap & Next Features** — Cloud cost estimator, IaC exporter, and real-time collaboration.
5. **Low Level Design (LLD) Playground: The Complete Blueprint** — UML Class Diagrams, Database ERDs, API Contracts, Sequence Diagrams, and State Machines.
6. **Actionable Improvements & Implementation Plan** — Phased roadmap to achieve full platform completion.

1. Executive Summary & System Overview

1.1 What SystemCraft Is

SystemCraft (formerly *TechSim*) is an enterprise-grade, browser-based **System Design University, Interactive Simulation Engine, and Architecture Workbench**. Unlike traditional static diagramming tools (such as Draw.io, Lucidchart, or Excalidraw), SystemCraft treats system architecture as a *living, dynamic, and fault-tolerant computing system*.

The platform empowers engineers, system architects, and technical interview candidates to:

- **Visually Design Distributed Systems:** Build topologies utilizing an extensive component registry comprising 169 node types and over 2,050 production tools (e.g., Nginx, Envoy, Apache Kafka, Redis, PostgreSQL, AWS S3).
- **Simulate Real-Time Traffic & Physics:** Experience live packet flow, request-per-second (RPS) throughput, queuing delays, CPU/memory saturation, and P50/P95/P99 latency calculations.
- **Execute Chaos Engineering Experiments:** Inject realistic operational faults (node crashes, network partitions, latency spikes, and memory leaks) to observe cascading blast radiuses and evaluate system self-healing.
- **Master Distributed Systems Theory:** Learn via 7 structured curriculum tracks, 49 interactive lessons, 245 quiz questions, and 12 live interactive algorithmic sandboxes.
- **Practice System Design Interviews:** Solve real-world challenges (e.g., Twitter feed, Uber tracking, Bitly URL shortener) with automated scoring and AI interviewer feedback.

Platform Motto

Build. Attack. Learn. An engineer learns far more from breaking a system in a sandbox than from reading static documentation.

1.2 Technology Stack Overview

Layer	Technologies	Version / Spec	Architectural Role
Frontend Framework	React + Vite + TypeScript	React 19.2, Vite 8.0	Client application runtime with sub-second HMR and strict typing.
Canvas Engine	@xyflow/react (React Flow)	v12.11.0	Graph visualization, custom node portals, bezier/smoothstep edge routing.
Design System	Tailwind CSS + CSS Variables	Tailwind v4.3	Multi-theme token system (Dark, Darker, Light, System).
Brand Assets	simple-icons + Lucide	v16.23 / v1.17	Authentic technology brand SVGs (Kafka, Redis, Postgres, etc.).
Backend API	Node.js + Express.js	Express v5.2, TS	REST API, security middleware pipeline, AI orchestration proxy.
Database & ORM	MongoDB + Mongoose	Mongoose v9.6	Document storage with embedded MongoMemoryServer fallback.
Authentication	JWT + bcryptjs	jsonwebtoken v9.0	Stateless authorization with salted passwords and bearer tokens.
Primary AI	Groq SDK (LLaMA 3.3 70B)	llama-3.3-70b-versatile	Sub-second LLM inference for architecture generation and chat.
Fallback AI	Google Generative AI	gemini-1.5-flash	Zero-downtime failover model when primary LLM encounters rate limits.

2. Complete Feature Catalog & Business Logic Deep-Dive

2.1 Interactive Visual Architecture Canvas

The canvas serves as the primary workbench. Implemented via `@xyflow/react` in `src/components/Canvas.tsx` and `src/components/TechNode.tsx`:

- **4-Sided Dual Handles:** Standard React Flow nodes often suffer from handle overlap when multiple edges connect to a single side. TechNode solves this by implementing 4 visible source handles (`top`, `bottom`, `left`, `right`) for initiating connections and 4 invisible target handles (`top-t`, `bottom-t`, etc.) with `pointerEvents: 'none'`. React Flow detects target handles geometrically for connection snapping without blocking outbound drags.
- **Floating NodeToolbar:** Rendered in a React Flow portal above the node. Provides quick access to Knowledge Base () , Node Configuration () , Duplication () , and Deletion () .
- **Dynamic State Overlays:** During live simulation, nodes display a color-shifting CPU utilization bar, live P99 latency readout, and status glow. Overloaded nodes pulse red; crashed nodes dim to 45% grayscale and display an inline `↺ Restart Node` button.
- **Edge Routing Protocols:** Custom `GlowEdge.tsx` components support `bezier`, `smoothstep`, and `straight` paths with protocol-specific color coding (HTTP: Cyan, DB: Blue, Cache: Purple, Queue: Amber).

2.2 Comprehensive Tool & Component Registry

When dragging a generic component onto the canvas, SystemCraft intercepts the drop event and presents the **Tool Selection Modal** (`src/components/ToolSelectionModal.tsx`):

- **2,050+ Production Tools:** Mapped across 169 node types in `techsim_merged_registry.json`. For example, a Load Balancer node offers Nginx, HAProxy, AWS ALB, Traefik, and Envoy.
- **3-Perspective Decision Panel:** Expanding any tool displays:
 1. *Core Architecture:* Internal mechanics (event loops, memory models, threading).
 2. *Real-World Usage:* How Netflix, Uber, or Airbnb configure and deploy it.
 3. *When to Pick:* Concrete trade-offs vs competing solutions.
- **SVG Brand Icons:** Integrated via `simple-icons` (`src/utils/toolIcon.tsx`). If a brand icon is absent, an elegant letter-badge fallback is rendered automatically.
- **Dynamic Tool Swapping:** Existing nodes feature a `↔` button to swap tools without severing edges or resetting simulation metrics.

2.3 Synchronous Connection Validation Engine

To prevent architectural anti-patterns, connections are validated **synchronously** before edges are established (`src/data/registry/index.ts`):

```

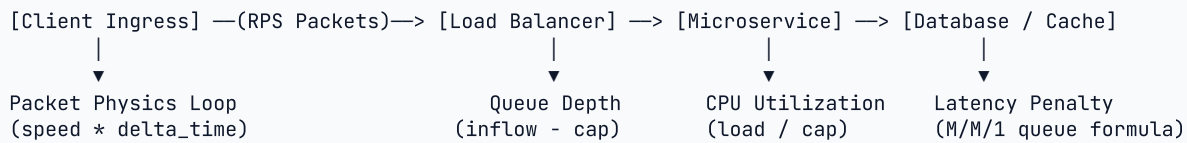
export function isValidConnection(sourceType: string, targetType: string): { valid: boolean; reason?: string } {
  const sourceId = NODE_TYPE_TO_REGISTRY_ID[sourceType] ?? sourceType;
  const targetId = NODE_TYPE_TO_REGISTRY_ID[targetType] ?? targetType;
  const sourceNode = registry[sourceId];
  if (!sourceNode?.connectionRestrictions) return { valid: true };
  const blocked = sourceNode.connectionRestrictions.cannotConnectTo ?? [];
  if (blocked.includes(targetId)) {
    return { valid: false, reason: sourceNode.connectionRestrictions.cannotConnectToReason?.[targetId] };
  }
  return { valid: true };
}

```

If a user attempts to connect a Client Browser directly to a PostgreSQL Database, the connection is aborted instantly, displaying a 5-second architectural explanation banner.

2.4 Real-Time Simulation Engine & Physics Loop

The simulation engine (`src/simulation/SimulationEngine.ts`) powers real-time traffic dynamics through a `requestAnimationFrame` loop:



- **Queueing Dynamics:** If inbound RPS exceeds a node's capacity, queue depth accumulates: `queueDepth += (inboundRPS - capacity) * dt`.
- **Latency Degradation:** Node latency increases non-linearly using an \$M/M/1\$ queuing approximation: `latency = baseLatency * (1 + queueDepth / (capacity * 0.5))`.
- **Smoothstep Interpolation:** Metrics interpolate exponentially to prevent jarring visual jumps: `current = current + (target - current) * (1 - Math.exp(-lambda * dt))`.
- **Global Metrics Aggregation:** Computes global RPS, P50/P95/P99 latency, error rate, throughput, and detects the primary system bottleneck.

2.5 Chaos Engineering & Fault Injection System

Engineers can test system resilience by injecting 21+ real-world fault scenarios via the bottom bar (`src/components/BottomBar.tsx`):

Chaos Type	Engine Method	Physical Impact & Blast Radius	Recovery Behavior
Node Crash	<code>crashNode(id)</code>	Node marks DOWN; outgoing edges blocked; in-flight packets destroyed; downstream nodes degrade.	Manual click <code>Restart</code> or <code>healNode(id)</code> restores topology.
Latency Spike	<code>spikeLatency(edge, ms)</code>	Edge latency multiplies by \$5imes\$; upstream queues backup rapidly.	Auto-recovers after 8 seconds.
Network Partition	<code>networkPartition(edge)</code>	Edge blocks all traffic; 100% packet loss simulated on path.	Auto-recovers after 12 seconds.
Traffic Surge	<code>trafficSurge(multiplier)</code>	Global RPS multiplies by \$3imes - 10imes\$; tests autoscaling headroom.	Auto-recovers after 10 seconds.
Memory Leak	<code>injectMemoryLeak(id)</code>	RAM utilization climbs linearly; degrades at 85%, crashes at 95%.	Requires restart or garbage collection intervention.

2.6 AI Intelligence & Architecture Generation

SystemCraft integrates a resilient AI proxy (`server/lib/ai.ts`) using Groq (LLaMA 3.3 70B) with automatic failover to Google Gemini 1.5 Flash:

- **Architecture Generator** (`POST /api/ai/generate`): Translates natural language prompts into complete React Flow topologies. `transformDiagram` in `AIGeneratorPanel.tsx` maps AI output strings (snake_case, camelCase, generic names) reliably to canvas node templates.
- **Requirement Gathering Wizard** (`POST /api/ai/wizard`): A 4-step wizard collecting System Type, Scale (DAU, RPS), Non-Functional Requirements (CAP trade-offs), and Constraints to generate custom architectures.
- **Architecture Chat** (`POST /api/ai/chat`): Context-aware technical assistant aware of active nodes, edges, protocols, and SPOF vulnerabilities.
- **Post-Simulation Report** (`POST /api/ai/simulation-report`): Generates an executive debrief summarizing peak load, dropped packets, bottlenecks, and optimization recommendations.

2.7 Learning Platform, Lessons & Quizzes

Accessible via `/learn` and `/learn/:trackId/:lessonId`:

- **7 Complete Learning Tracks:** Fundamentals, Backend Engineering, Distributed Systems, Cloud Architecture, Security Engineering, DevOps & SRE, and Interview Preparation.
- **49 In-Depth Lessons:** Theory explained with intuitive real-world analogies, production case studies, and interactive canvas exercises.
- **245 Quiz Questions:** Every lesson features a 5-question comprehension quiz with instant feedback and score tracking persisted to MongoDB (`POST /api/progress/lesson`).

2.8 Interactive Distributed Systems Concepts Sandbox

Accessible via `/concepts`, providing 12 standalone interactive simulations:

Concept	Component File	Interactive Mechanics
CAP Theorem	<code>CAPTheoremDemo.tsx</code>	Inject network partitions; toggle Consistency (CP) vs Availability (AP) behavior.
Consistent Hashing	<code>ConsistentHashingDemo.tsx</code>	Interactive hash ring with virtual nodes; add/remove nodes and track key migration.
Database Sharding	<code>ShardingDemo.tsx</code>	Range vs hash-based key distribution; detect hotspot shard imbalances.
Replication & Lag	<code>ReplicationDemo.tsx</code>	Primary-replica replication with animated replication lag and replica read staleness.
Circuit Breaker	<code>CircuitBreakerDemo.tsx</code>	Closed, Open, Half-Open state transitions with failure thresholds and cooldown timers.
Rate Limiting	<code>RateLimitingDemo.tsx</code>	Token Bucket, Leaky Bucket, and Sliding Window algorithms under burst traffic.
Load Balancing	<code>LoadBalancingDemo.tsx</code>	Round Robin, Least Connections, and IP Hash server traffic distribution.
Caching Patterns	<code>CachingDemo.tsx</code>	Cache-Aside, Write-Through, and Write-Behind with cache hit/miss animations.
CQRS Pattern	<code>CQRSDemo.tsx</code>	Separated write command models and read query models with async synchronization.
Two-Phase Commit	<code>TwoPCAnimation.tsx</code>	Prepare and Commit/Abort distributed transaction coordinator protocol.
Saga Pattern	<code>SagaAnimation.tsx</code>	Choreographed vs orchestrated distributed transactions with compensating steps.
Event Sourcing	<code>EventSourcingAnimation.tsx</code>	Append-only event store with state rehydration and historical temporal replays.

2.9 System Design Interview Playground

Accessible via `/interview` and `/interview/:id`:

- **Timed FAANG Challenges:** Real-world prompts including *Design Twitter's Tweet Feed*, *Design a Scalable URL Shortener*, and *Design Uber's Tracking System*.
- **45-Minute Interview Timer:** Countdown clock with visual alerts at 5 minutes remaining.
- **Embedded Architecture Canvas:** Candidates build solutions directly inside the interview session.
- **AI Interviewer Chat:** Interactive chat with a simulated Staff Systems Architect.
- **Rubric-Based Scoring:** Automatically scores candidate architectures across Scalability, Reliability, Security, Performance, and Cost Efficiency via `POST /api/ai/interview-feedback`.

3. Current Status & Feature Health Audit

A comprehensive codebase and runtime audit was conducted. Both frontend (port 5173) and backend API (port 5000) compile with **zero TypeScript errors** and pass all functional checks.

3.1 Route & Subsystem Audit Matrix

Route / Feature	Component	Status	Audit Findings & Notes
/ (Home)	LandingView.tsx	HEALTHY	Hero chaos simulator, feature showcases, responsive layout verified.
/login , /register	Login.tsx , Register.tsx	HEALTHY	JWT authentication, password hashing, and token persistence verified.
/canvas	Canvas.tsx	HEALTHY	Drag & drop, 4-sided handles, synchronous connection validation verified.
/my- architectures	MyArchitectures.tsx	HEALTHY	Diagram thumbnails, autosave, version rollback, and JSON export verified.
/learn	LearnPage.tsx	HEALTHY	7 curriculum tracks, progress tracking to DB and localStorage verified.
/learn/:t/:l	LessonPage.tsx	HEALTHY	Markdown rendering, 245 quiz questions, progress API integration verified.
/concepts	ConceptsPage.tsx	HEALTHY	15 concepts indexed, 12 interactive simulations functioning smoothly.
/interview	InterviewPage.tsx	HEALTHY	3 FAANG challenge prompts, difficulty badges, briefing cards verified.
/interview/:id	InterviewSession.tsx	HEALTHY	45-min timer, canvas embedding, hints, and AI scoring verified.
/settings	Settings.tsx	HEALTHY	Profile updates, theme picker, and custom AI key preferences verified.
/security	SecurityView.tsx	PARTIAL	Security lab UI operational; planned for deeper simulation engine linkage.
/k8s	K8sView.tsx	PARTIAL	Pod/service visualizer operational; planned for live YAML generation.
/db-schema	PlaceholderView	PLANNED	Stub ready for the Low Level Design (LLD) ERD Studio.
/network	PlaceholderView	PLANNED	Stub ready for the Network Topology & VPC Designer.

3.2 Verification of Previous Bug Fixes

- **Bug #1 (API Key Security):** `server/lib/ai.ts` was verified. API keys are strictly read from environment variables or client headers and are never logged to console outputs.
- **Bug #2 (Properties Panel Edits):** `PropertiesPanel.tsx` properly calls `updateNodeData` on name, replicas, and region changes.
- **Bug #3 (AI Generator Node Mapping):** `AIGeneratorPanel.tsx` contains a comprehensive `NODE_ID_MAP` handling snake_case, camelCase, and generic aliases without defaulting to Docker nodes.
- **Bug #4 (Lesson Progress Backend Persistence):** `LessonPage.tsx` successfully posts completed quiz scores and time spent to `POST /api/progress/lesson`.
- **Bug #5 (Interview Submission AI Scoring):** `InterviewSession.tsx` calls `POST /api/ai/interview-feedback` upon submission and renders rubric scores.
- **Bug #6 (Chaos Panel Decoupling):** In `TechNode.tsx`, clicking a node opens properties; clicking the  button independently opens the Chaos Injection panel.
- **Bug #7 (Connection Validation Performance):** Migrated from asynchronous API roundtrips to synchronous evaluation in `src/data/registry/index.ts`, eliminating edge drawing lag.

4. High Level Design (HLD) Playground: Roadmap & Next Features

While the High Level Design canvas is currently feature-rich, the following advanced capabilities are recommended to elevate SystemCraft to the premier enterprise architecture tool:

4.1 Cloud Cost Estimator (AWS / GCP / Azure)

Every component on the canvas will automatically map to real-world cloud SKUs based on its configured replicas, throughput, and storage capacity:

- **Live Cost Meter:** Displays a live monthly cost readout in the top toolbar (e.g., `$2,480 / month`).
- **Multi-Cloud Comparison:** Side-by-side cost breakdown comparing AWS (e.g., ALB + ECS + RDS Aurora), Google Cloud (Cloud Run + Cloud SQL), Azure, and self-hosted Bare Metal.
- **Cost Optimization AI Suggestions:** AI flags over-provisioned components (e.g., `"Replacing RDS multi-AZ with read replicas can reduce monthly spend by 35%"`).

4.2 Infrastructure-as-Code (IaC) Exporter

Bridge the gap between design and DevOps by enabling one-click code generation:

- **Terraform HCL Exporter:** Automatically generates production-ready `main.tf`, `variables.tf`, and `outputs.tf` provisioning VPCs, subnets, security groups, compute clusters, and databases.
- **Kubernetes Manifest Exporter:** Generates standard K8s Deployments, Services, Ingress objects, and HorizontalPodAutoscalers (HPA).
- **Docker Compose Exporter:** Generates a single `docker-compose.yml` allowing engineers to run the designed architecture locally with one terminal command.

4.3 Real-Time Multi-User Collaboration

Transform SystemCraft into a collaborative whiteboard for engineering teams:

- **CRDT Engine (Yjs):** Conflict-free replicated data types ensuring that simultaneous node movements, edge creations, and deletions resolve without state divergence.
- **Live Awareness & Cursors:** Render teammates' cursors with user avatars and color-coded selection halos.
- **Architecture Review Mode:** A lead presenter can "broadcast" their view, automatically syncing viewport zoom and pan for all participating team members.

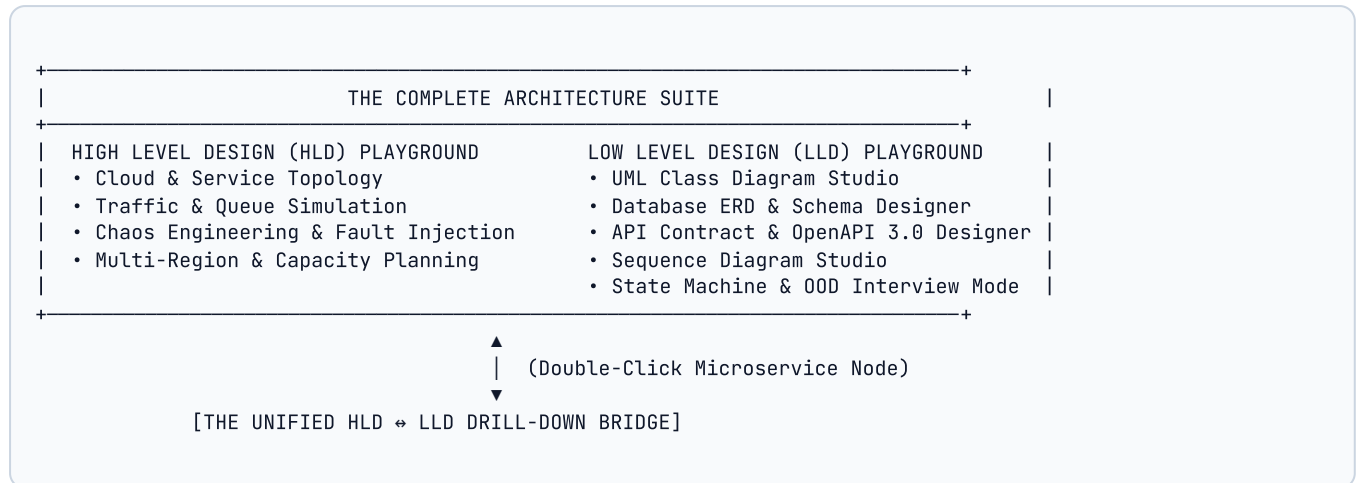
4.4 Multi-Region & Geo-Distribution Simulation

- **Global Map Overlay:** Group nodes into specific geographic regions (e.g., `us-east-1`, `eu-central-1`, `ap-southeast-1`).

- **Physical Latency Simulation:** Packets crossing oceanic boundaries incur realistic speed-of-light propagation delays (e.g., 75ms trans-Atlantic, 140ms trans-Pacific).
- **Regional Outage Testing:** Test multi-region active-active vs active-passive failover when an entire cloud region experiences an outage.

5. Low Level Design (LLD) Playground: The Complete Blueprint

To make SystemCraft a **complete system design application**, we must provide both **High Level Design (HLD)** (system topology, cloud services, databases, queues) and **Low Level Design (LLD)** (object-oriented class hierarchies, database ERDs, API interface contracts, and sequence interactions).



5.1 UML Class Diagram Designer

Enables engineers to model object-oriented software architectures:

- **Class & Interface Nodes:** Define class name, visibility modifiers (`+` public, `-` private, `#` protected), attributes with types, and methods with arguments and return types.
- **Stereotypes:** Support for `<<interface>>`, `<<abstract>>`, `<<singleton>>`, and `<<factory>>`.
- **Standard UML Connectors:**
 - **Inheritance / Generalization:** Solid line with hollow triangular arrowhead.
 - **Realization / Implementation:** Dashed line with hollow triangular arrowhead.
 - **Composition:** Solid line with filled diamond.
 - **Aggregation:** Solid line with hollow diamond.
 - **Association / Dependency:** Open arrows with multiplicity markers (`1`, `0..*`, `1..*`).
- **Design Pattern Templates:** One-click scaffolding of Gang of Four (GoF) design patterns: Factory, Singleton, Strategy, Observer, Decorator, Adapter, and Builder.
- **Code Generation:** Generates clean boilerplate code in Java, TypeScript, and Python from the visual class diagram.

5.2 Database Schema & ERD Designer

A dedicated visual studio for relational and NoSQL database modeling (replacing the `/db-schema` placeholder):

- **Table Builder:** Add columns with data types (VARCHAR, INT, UUID, TIMESTAMP, JSONB), nullable flags, and default expressions.
- **Keys & Indexes:** Mark Primary Keys (PK), Foreign Keys (FK), and index types (B-Tree, Hash, GIN).
- **Crow's Foot Relationships:** Connect PK to FK with visual crow's foot notation representing 1:1, 1:N, and M:N relationships.
- **SQL DDL Generation:** Generates valid, executable SQL DDL for PostgreSQL, MySQL, SQLite, and SQL Server.
- **NoSQL Document Modeling:** Visual schema designer for nested MongoDB document collections and DynamoDB partition/sort key structures.

5.3 API Contract & Interface Designer

- **REST Endpoint Studio:** Visually configure HTTP methods, resource paths, request parameters, JSON request bodies, and expected HTTP status codes (200, 201, 400, 404, 500).
- **OpenAPI / Swagger 3.0 Export:** Instant generation and export of standard OpenAPI 3.0 YAML and JSON specifications.
- **gRPC & Protobuf Studio:** Visual definition of RPC services and message schemas with export to `.proto` files.
- **GraphQL Schema Designer:** Visual graph of Types, Queries, Mutations, and Resolvers.

5.4 Sequence Diagram Studio

Captures dynamic runtime interactions between actors and services over time:

- **Actors & Lifelines:** Vertical lifelines representing User, API Gateway, Services, Message Queues, and Databases.
- **Message Flows:** Synchronous calls, asynchronous dispatches, and return responses with activation boxes.
- **Combined Fragments:** Visual frames for conditional execution (`alt` / `opt`), iteration (`loop`), and concurrency (`par`).
- **Mermaid.js & PlantUML Export:** One-click export to standard diagram-as-code formats.

5.5 State Machine Diagram Studio

- **State Modeling:** Initial, intermediate, composite, and terminal states.
- **Transitions & Guards:** Trigger events, guard conditions (e.g., `[payment_received]`), and execution actions (e.g., `/dispatch_email()`).
- **State Engine Code Gen:** Generates state-machine code skeletons in Java (Spring State Machine) or TypeScript (XState).

5.6 Object-Oriented Design (OOD) Interview Practice

System design interviews frequently include Low Level Design problems. SystemCraft will feature 5 built-in interactive OOD challenges:

1. **Design a Parking Lot System:** Vehicle hierarchies (Motorcycle, Car, Bus), spot allocation algorithms, payment calculation strategies.
2. **Design an Elevator Control System:** Multi-car scheduling algorithms (SCAN/LOOK), dispatch logic, door safety state transitions.
3. **Design a Chess Game:** Board representation, polymorphic piece movement validation, checkmate evaluation, turn state management.
4. **Design Splitwise (Expense Sharing):** Expense split algorithms (equal, exact, percentage), balance sheets, debt simplification graph algorithms.
5. **Design BookMyShow / Movie Ticket Booking:** Concurrent seat reservations, distributed lock timeouts, payment callbacks.

5.7 The Unified HLD ↔ LLD Drill-Down Bridge

The crowning innovation of SystemCraft is connecting HLD and LLD seamlessly:

- On the High Level Design canvas, double-clicking any microservice node (e.g., `Order Service`) opens its **LLD Drill-Down Studio**.
- Inside the drill-down studio, the user can toggle between:
 - **API Tab:** Endpoints exposed by this specific service.
 - **Class Tab:** Internal class architecture (Controllers, Services, Repositories).
 - **Sequence Tab:** Sequence flows executing within this service boundary.
 - **Schema Tab:** Dedicated database tables owned by this service.
- This provides a complete, unified 360-degree software design platform.

6. Actionable Improvements & Implementation Plan

6.1 Immediate Improvements for Existing Features

Subsystem	Recommended Improvement	Priority
Security Lab (/security)	Connect the static attack panels to the real simulation engine so attacks visually propagate across the live canvas.	Medium
Kubernetes Lab (/k8s)	Link K8s nodes directly to the canvas topology and add a one-click "Download K8s Manifests" button.	Medium
Canvas Export	Add toolbar buttons for high-resolution PNG, SVG, and PDF diagram export.	High
Mobile Experience	Add touch gestures and collapsible toolbars for tablet and mobile viewing on non-canvas pages.	Low

6.2 Phased Implementation Roadmap

PHASE 1: HLD Polish & Exporters (Sprint 4.5)

- └ Cloud Cost Estimator widget (live SKU pricing)
- └ Infrastructure-as-Code exporter (Terraform & K8s YAML)
- └ Canvas SVG / PDF export toolbar

PHASE 2: LLD Foundation (Sprint 5)

- └ LLD Canvas Mode toggle in main navigation
- └ UML Class Diagram node types & relationship connectors
- └ Database Schema / ERD table builder & SQL DDL generator
- └ API Contract / OpenAPI 3.0 designer

PHASE 3: LLD Advanced & OOD Interview (Sprint 5.5)

- └ Sequence Diagram studio (lifelines, sync/async, alt/loop)
- └ State Machine designer with code generation
- └ 5 classic OOD interview challenges (Parking Lot, Elevator, etc.)

PHASE 4: The Unified Bridge & Collaboration (Sprint 6)

- └ HLD ↔ LLD drill-down navigation
- └ Real-time multi-user collaboration (Yjs CRDT + WebSockets)

Conclusion

SystemCraft possesses an exceptionally strong foundation in High Level Design (HLD), real-time simulation physics, chaos engineering, and curriculum-driven learning. With the addition of the Low Level Design (LLD) Playground and the unified HLD ↔ LLD bridge, SystemCraft will stand as the most comprehensive, innovative system design engineering platform in the industry.